ENTREGABLE 08 / DOCUMENTACIÓN FINAL

NutriBalance

Documento final del escenario NutriTurnos

NutriBalance es la aplicación del escenario NutriTurnos del TP Integrador de Desarrollo Web 2026. La web pública permite consultar disponibilidad y solicitar un turno. El profesional ingresa al panel para gestionar pacientes, confirmar turnos y registrar consultas. La demostración emplea datos ficticios.

Stack y arquitectura

CAPA	TECNOLOGÍA Y FUNCIÓN
Interfaz	React, TypeScript, Vinext/Vite y CSS propio; fuentes locales Manrope y Source Serif 4.
Web publicada	Cloudflare Worker mediante Sites; páginas públicas y gateway **/api/v1/***.
Backend	API Express externa en **api.neadev.com**; sesiones, disponibilidad, pacientes, turnos y consultas.
Datos	PostgreSQL según el contrato del backend. El esquema físico no está en este repositorio.
Código y despliegue	GitHub privado, **Dockerfile** y **render.yaml** preparados para Render. La URL de Render aún no está publicada.

Decisiones principales

- El sitio es público, pero cada operación clínica requiere una sesión profesional autorizada por Express. El navegador usa un gateway del mismo origen que reenvía sólo rutas permitidas, la cookie de sesión y el token CSRF.
- Una solicitud crea un **turno pendiente** y un comprobante. La ficha clínica se crea o vincula de forma explícita al confirmar; los datos de contacto no se convierten automáticamente en historia clínica.
- La disponibilidad usa horarios de Argentina y envía a la API el instante exacto elegido. Ante un conflicto de horario, la interfaz muestra el error y no repite una creación de forma automática.
- Las pantallas de demostración y el respaldo D1 se mantienen separados del flujo principal conectado a Express. Sus datos no acreditan persistencia en PostgreSQL.
- La identidad visual usa tipografía local, colores verde bosque y marfil, diseño adaptable y estados de foco visibles. Las páginas privadas se excluyen de la indexación.

Dificultades y resolución

La integración con Express exigió coordinar el origen permitido, las cookies de sesión y CSRF. Se implementó el gateway **/api/v1/*** para mantener la sesión en el origen de la web y se probó el rechazo de orígenes ajenos. También se distinguió la solicitud pendiente de un turno confirmado, para no presentar un pedido como atención ya reservada.

La API pública de PageSpeed agotó su cuota (HTTP 429). Se ejecutó Lighthouse localmente sobre la URL publicada y se guardó el reporte completo en **07-evidencias-seo-calidad.md**. Docker Desktop no arrancó en el equipo de prueba; se verificaron la compilación independiente de Node y sus rutas, pero la imagen Docker queda por construir en Render.

Tareas y uso de IA

Según la distribución informada por el equipo, **Mauricio Pelozo** se encargó del frontend y **Bruno Cammors** del backend Express y la base de datos. La documentación y las pruebas de este repositorio acompañan el trabajo del frontend. Se usó **Codex** como apoyo para implementación, revisión de UX, pruebas y redacción; el historial Git identifica commits de la cuenta del proyecto y de Codex. No se atribuye a Codex trabajo del backend ni se presume uso de IA por parte de Bruno.

Resultado y límites de la evidencia

La revisión del frontend aprobó TypeScript, la compilación Node para Render y **18 pruebas** de dominio, cliente HTTP y gateway. La auditoría móvil de Lighthouse obtuvo **77 en rendimiento, 100 en accesibilidad, 81 en buenas prácticas y 100 en SEO** en una ejecución. La medición y sus oportunidades de mejora están en el entregable **07-evidencias-seo-calidad.md**. El análisis funcional **01-analisis-funcional.md** define el alcance y los criterios del flujo.

La inspección directa de PostgreSQL, una prueba en teléfono físico y el despliegue en Render quedan fuera de esta evidencia. El repositorio GitHub es privado; quien evalúe el código necesita acceso.